

Yes. **The version I described is close to what a startup or enterprise team would build**, and it would likely take **4–8 months** for one person to complete well. For a final-year project or portfolio project, it's probably too ambitious.

I recommend reducing the scope while keeping the parts that make it impressive.

Simplified Project

Project Name

KubeAssist AI – Kubernetes Troubleshooting Assistant

What is the project?

KubeAssist AI is a web application that helps developers troubleshoot Kubernetes cluster issues using AI.

Instead of manually checking logs, events, and documentation, users can either ask questions or select a pod, and the application automatically gathers relevant information from the cluster. It then retrieves relevant Kubernetes documentation using RAG and generates an explanation along with possible solutions.

The goal is **not to automate SRE work completely**, but to **make Kubernetes troubleshooting faster and easier**.

Scope

Instead of covering the entire cluster, focus on:

- Pods
- Deployments
- Services
- Events

- Logs

That's enough to demonstrate the concept without becoming overwhelming.

Features

1. Cluster Dashboard

Show

- Pods
- Status
- Restarts
- Namespace

Example

payment-service

Running

0 Restarts

or

auth-service

CrashLoopBackOff

12 Restarts

2. Pod Details

Clicking a pod shows

- Pod status
- Events
- Logs
- YAML configuration

3. AI Troubleshooting

User asks

Why is this pod crashing?

System automatically collects

- Logs
- Events
- Pod configuration

Then retrieves Kubernetes documentation using RAG.

Finally

Possible Cause

Liveness probe is failing.

Reason

The application takes 30 seconds to start while the probe begins after 5 seconds.

Suggested Fix

Increase initialDelaySeconds.

4. Documentation Search

Instead of Google,

the AI searches embedded Kubernetes documentation.

Example

What is CrashLoopBackOff?

or

How does a readiness probe work?

5. Log Summarization

Instead of showing 500 log lines,

AI summarizes them.

Example

Main Errors

Database connection refused

Redis timeout

Application restarted 12 times

6. Configuration Checker

Upload a Deployment YAML.

The application checks for common mistakes.

Example

No resource limits

No liveness probe

Using latest image tag

Service selector mismatch

Tech Stack

Frontend

- React
 - TypeScript
 - Tailwind CSS
-

Backend

Go

Responsibilities

- Connect to Kubernetes
- Read Pods
- Read Logs
- Read Events
- Read YAML

Why Go?

Because Kubernetes itself is written in Go, so using `client-go` is a practical skill to demonstrate.

AI Backend

Python

Responsibilities

- RAG
 - LLM interaction
 - Log summarization
 - Troubleshooting
-

Database

PostgreSQL

Store

- Chat history
 - Uploaded YAML
 - Investigation history
-

Vector Database

FAISS

Store

- Kubernetes documentation
-

Components

Frontend

Dashboard

↓

Go Backend

Collects

- Pods
- Logs
- Events
- YAML

↓

Python AI Service

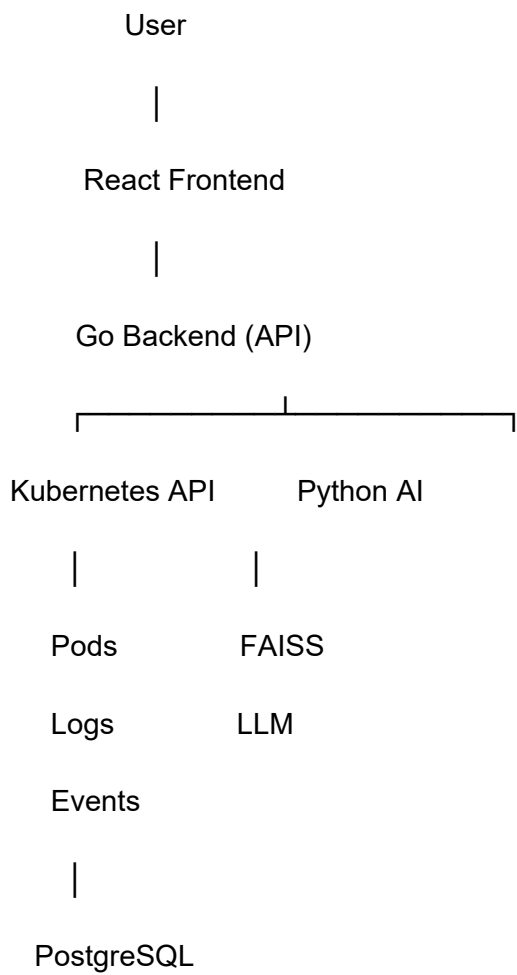
Retrieves documentation

↓

LLM

↓

System Design



Data Flow

User selects Pod



Go Backend

↓

Fetch Logs

↓

Fetch Events

↓

Fetch YAML

↓

Send Context

↓

Python AI

↓

Retrieve Kubernetes Docs

↓

Generate Explanation

↓

Return Answer

What You'll Learn

- Kubernetes fundamentals
- Go (`client-go`)
- REST APIs
- Python AI integration
- RAG pipeline
- Vector databases

- React frontend
 - Docker
 - Basic system design
-

Time Estimate

- **Frontend:** 1–2 weeks
- **Go backend:** 2–3 weeks
- **RAG pipeline:** 1–2 weeks
- **Testing and polish:** 1–2 weeks

Total: Around **6–8 weeks** of part-time work, which is much more manageable for a student.

Why I prefer this version

It still demonstrates:

- Go and Kubernetes (`client-go`)
- AI/RAG concepts
- Full-stack development
- System design
- Real-world troubleshooting

But it avoids advanced production concerns like distributed agents, message queues, Prometheus integration, Loki, multi-cluster management, and autonomous remediation. Those can always be added later as future enhancements if you continue developing the project. This gives you a polished, complete project rather than a very large project that's difficult to finish.